



UNIVERSIDAD CATOLICA DEL NORTE

FACULTAD DE INGENIERIA Y CIENCIAS GEOLOGICAS

DEPARTAMENTO DE INGENIERIA DE SISTEMAS Y COMPUTACION

**MODELOS DE LENGUAJE LOCALES FRENTE A METODOS
ALGORITMICOS EN LA EXTRACCION DE NARRATIVAS
COHERENTES**

*Trabajo de Titulacion Modalidad Capstone Project para optar al titulo de Ingeniero Civil en
Computacion e Informatica*

SEBASTIÁN IGNACIO CONCHA MACÍAS

Profesor Guia: PhD. Brian Keith Norambuena

Antofagasta, Chile

2026

Dedicatoria

Agradecimientos

Indice de contenidos

<i>CAPITULO I: INTRODUCCION</i>	<i>XI</i>
1.1. Descripcion del problema	XI
1.2. Justificacion	XII
1.3. Marco teorico	XII
1.3.1. Extraccion de narrativas basada en eventos	XII
1.3.2. Modelos de lenguaje de gran escala (LLMs)	XII
1.3.3. Evaluacion LLM-as-a-judge	XII
1.3.4. Escalabilidad en extraccion narrativa	XIII
1.4. Objetivos	XIII
1.4.1. Objetivo general	XIII
1.4.2. Objetivos especificos	XIII
1.4.3. Resultados esperados	XIII
1.5. Metodologia	XIV
1.6. Contenido del documento	XIV
<i>CAPITULO II: REVISION BIBLIOGRAFICA Y SELECCION DE MODELOS</i>	<i>XVI</i>
2.1. Revision bibliografica	XVI
2.1.1. Extraccion de narrativas basada en eventos	XVI
2.1.2. Uso de LLMs en tareas narrativas	XVII
2.1.3. Evaluacion automatica con LLM-as-a-judge	XVII
2.2. Seleccion de modelos de lenguaje locales	XVIII
<i>CAPITULO III: DISEÑO E IMPLEMENTACION DEL PIPELINE EXPERIMENTAL</i>	<i>XIX</i>
3.1. Arquitectura del pipeline	XIX
3.2. Componentes del sistema	XX
3.2.1. Modulo de scoring	XX
3.2.2. Modulo de extraccion	XXI
3.2.3. Modulo de evaluacion	XXI
3.2.4. Registro de metricas	XXI
3.3. Dataset	XXII
3.3.1. Corpus completo	XXII
3.3.2. Dataset experimental	XXII
3.4. Diseño experimental	XXII
3.4.1. Combinaciones evaluadas	XXII
3.4.2. Parametros experimentales	XXIII
3.4.3. Metricas	XXIII
3.4.4. Evaluacion estadistica	XXIV
3.5. Infraestructura	XXIV
<i>CAPITULO IV: EXPERIMENTOS Y RESULTADOS PRELIMINARES</i>	<i>XXV</i>
4.1. Ejecucion de experimentos	XXV
4.2. Resultados de coherencia	XXV
4.2.1. Estadisticas descriptivas	XXV

4.2.2. Efecto del tamaño de narrativa.....	XXVI
4.2.3. Narrative Trails vs. Narrative Maps	XXVII
4.2.4. Robustez entre segmentos del corpus.....	XXVIII
4.3. Comparaciones estadísticas	XXIX
4.4. Evaluación LLM-as-a-judge	XXXII
4.5. Análisis de trade-offs calidad vs. eficiencia	XXXIII
<i>CAPÍTULO V: DISCUSIONES PRELIMINARES.....</i>	<i>XXXV</i>
5.1. Estado de cumplimiento de los objetivos	XXXV
5.2. Hallazgos principales.....	XXXV
5.3. Dificultades encontradas	XXXVI
5.4. Trabajo restante.....	XXXVI
<i>Referencias Bibliográficas.....</i>	<i>XXXVII</i>
<i>Anexo A: Estructura del Proyecto y Formato de Resultados.....</i>	<i>XXXVIII</i>
<i>A.1. Estructura del repositorio</i>	<i>XXXVIII</i>
<i>A.2. Formato de resultados</i>	<i>XXXVIII</i>

Indice de figuras

FIGURA 1. ARQUITECTURA GENERAL DEL PIPELINE EXPERIMENTAL.....	XX
FIGURA 2. BOX PLOT DE COHERENCIA PROMEDIO POR METODO (384 EXPERIMENTOS).	XXVI
FIGURA 3. COHERENCIA POR METODO Y TAMANO DE NARRATIVA (N=6, N=12, N=18).	XXVII
FIGURA 4. EFECTO DEL TAMANO DE NARRATIVA EN LA COHERENCIA.....	XXVII
FIGURA 5. . TRAILS VS. MAPS POR SCORER Y TAMANO DE NARRATIVA.	XXVIII
FIGURA 6. ROBUSTEZ: COHERENCIA POR PUNTO DE INICIO EN EL CORPUS.....	XXIX
FIGURA 7. HEATMAP DE TAMANO DE EFECTO (COHEN'S D) ENTRE PARES DE METODOS.....	XXXI
FIGURA 8. BOX PLOT DE JUDGE SCORES POR METODO.	XXXII
FIGURA 9. DIAGRAMA DE CALIDAD VS. TIEMPO DE EJECUCION.	XXXIV

Indice de tablas

TABLA 1. OBJETIVOS ESPECIFICOS Y RESULTADOS ESPERADOS.....	XIII
TABLA 2. MODELOS CANDIDATOS EVALUADOS.....	XVIII
TABLA 3. METODOS EVALUADOS.....	XXII
TABLA 4. PARAMETROS EXPERIMENTALES.....	XXIII
TABLA 5. METRICAS REGISTRADAS.....	XXIII
TABLA 6. INFRAESTRUCTURA UTILIZADA.....	XXIV
TABLA 7. RANKING DE METODOS POR COHERENCIA PROMEDIO.....	XXV
TABLA 8. COHERENCIA PROMEDIO POR METODO Y TAMANO DE NARRATIVA.....	XXVI
TABLA 9. COMPARACION TRAILS VS. MAPS POR SCORER (COHERENCIA MEDIA GLOBAL).....	XXVIII
TABLA 10. COMPARACIONES SIGNIFICATIVAS EN COHERENCIA ($P_{\text{BONFERRONI}} < 0.05$).....	XXIX
TABLA 11. COMPARACIONES NO SIGNIFICATIVAS ($P_{\text{BONFERRONI}} \geq 0.05$).....	XXX
TABLA 12. JUDGE SCORES PROMEDIO POR METODO.....	XXXII
TABLA 13. TRADE-OFFS CALIDAD VS. EFICIENCIA.....	XXXIII
TABLA 14. ESTADO DE CUMPLIMIENTO DE LOS OBJETIVOS ESPECIFICOS.....	XXXV

Nomenclatura

API: Application Programming Interface.

GPU: Graphics Processing Unit.

LP: Linear Programming.

LLM: Large Language Model.

NLP: Natural Language Processing.

RAM: Random Access Memory.

VRAM: Video Random Access Memory.

Glosario

Coherencia narrativa: Grado en que una secuencia de documentos forma una historia logica y temporalmente consistente, medida como el promedio de las puntuaciones de coherencia entre pares consecutivos de documentos.

Cohen's d: Medida estadística del tamaño del efecto que cuantifica la diferencia entre dos medias en unidades de desviación estándar.

Embeddings: Representaciones vectoriales densas de texto en un espacio de alta dimensionalidad, utilizadas para calcular similitud semántica entre documentos.

Extracción de narrativas basada en eventos: Tarea de identificar y organizar secuencias de documentos que representan historias coherentes dentro de colecciones documentales, bajo el supuesto de correspondencia uno a uno entre documentos y eventos.

LLM-as-a-judge: Protocolo de evaluación automática que utiliza un modelo de lenguaje como evaluador de calidad, asignando puntuaciones de coherencia a secuencias documentales.

Narrative Maps: Método de extracción narrativa que formula la selección de documentos como un problema de programación lineal entera y lo resuelve mediante una relajación de programación lineal fraccional (LP relaxation), seleccionando un subgrafo de documentos que maximiza la coherencia

Narrative Trails: Método de extracción narrativa basado en el algoritmo MaxiMin Dijkstra que busca el camino que maximiza la mínima coherencia entre aristas consecutivas.

Ollama: Framework de ejecución local de modelos de lenguaje que permite la inferencia sin depender de servicios en la nube.

Scorer: Componente del pipeline encargado de calcular la puntuación de coherencia entre pares de documentos.

Sentence-Transformers: Biblioteca de Python que proporciona modelos preentrenados para generar embeddings de oraciones, utilizada como baseline algorítmico en este trabajo.

Test-t de Welch: Prueba estadística para comparar medias de dos grupos cuando no se asume igualdad de varianzas.

Resumen

El presente informe describe el proyecto de investigación orientado a evaluar el desempeño de modelos de lenguaje de código abierto ejecutados localmente en tareas de extracción de narrativas basada en eventos, entendida como la identificación y organización de secuencias de documentos que conforman historias coherentes dentro de colecciones documentales.

La motivación principal radica en la falta de evidencia empírica sistemática respecto al desempeño de modelos locales en este dominio, en contraste con los modelos propietarios y los métodos algorítmicos tradicionales que han dominado la literatura, una brecha especialmente relevante en contextos donde el uso de servicios comerciales no resulta factible por razones de costo, dependencia tecnológica o restricciones de acceso a los datos. El proyecto compara modelos de lenguaje locales (LLaMA 3.2 y Mistral 7B, ejecutados vía Ollama) contra métodos algorítmicos tradicionales (Narrative Maps, basado en una relajación de programación lineal fraccional (LP relaxation), y Narrative Trails, basado en el algoritmo MaxiMin Dijkstra) y un modelo propietario de referencia (GPT-4o-mini), utilizando un corpus experimental de 931 artículos noticiosos chilenos. Para ello se diseñó e implementó un pipeline modular en Python que integra los componentes de scoring de coherencia, extracción narrativa, evaluación mediante el protocolo LLM-as-a-judge y registro de métricas de eficiencia computacional. El diseño experimental contempla ocho combinaciones de algoritmo y scorer, tres tamaños de narrativa ($n=6, 12, 18$), cuatro puntos de inicio en el corpus y cinco repeticiones por configuración para los métodos basados en LLM, totalizando 384 experimentos ejecutados en Google Colab con GPU NVIDIA A100.

A la fecha se ha completado la revisión bibliográfica, la selección de modelos, el diseño e implementación del pipeline y la ejecución del conjunto completo de experimentos, junto con su análisis estadístico mediante test-t de Welch, corrección de Bonferroni para comparaciones múltiples y cálculo de Cohen's d para el tamaño del efecto. Los resultados indican que los métodos algorítmicos tradicionales superan a los modelos de lenguaje locales en coherencia narrativa medida por `avg_edge_coherence`, con diferencias estadísticamente significativas ($p < 0.001$) y tamaños de efecto grandes (Cohen's $d > 2$). Narrative Trails supera consistentemente a Narrative Maps como algoritmo de extracción, independientemente del scorer utilizado, con una ventaja de entre +0.11 y +0.16 puntos de coherencia media. El scorer algorítmico basado en embeddings (Sentence-Transformers) obtiene la mayor coherencia y la menor variabilidad, seguido por el modelo propietario GPT-4o-mini, mientras que los modelos locales presentan coherencia significativamente menor y alta variabilidad entre segmentos del corpus, con una degradación pronunciada al aumentar el tamaño de la narrativa. Sin embargo, el protocolo LLM-as-a-judge asigna puntuaciones similares (7-8/10) a todos los métodos, lo que sugiere una divergencia entre la métrica matemática de coherencia y la evaluación holística del juez. Las etapas restantes del proyecto incluyen el análisis detallado de esta divergencia, la discusión de sus implicaciones para el campo y la redacción del informe final.

CAPITULO I: INTRODUCCION

El presente capitulo describe el contexto del proyecto, incluyendo la problematica, justificacion y solucion propuesta. Se exponen los objetivos generales y especificos, junto con los resultados esperados. Ademas, se detalla la metodologia de trabajo que permite visualizar como se lleva a cabo la investigacion.

1.1. Descripcion del problema

La extraccion de narrativas es definida como la tarea de identificar y organizar secuencias de documentos que relatan historias con sentido logico dentro de grandes colecciones de datos. Esta labor es fundamental para los procesos de comprension y analisis de informacion (sensemaking) en entornos con alta densidad documental y cae dentro del paraguas general del campo de information retrieval (Keith et al., 2023).

Tradicionalmente, el problema de extraccion de narrativas ha sido abordado mediante tecnicas algoritmicas basadas en optimizacion. Algunos enfoques para resolver este problema incluyen Narrative Maps (Keith & Mitra, 2021), que utiliza programacion lineal para modelar coherencia narrativa, y Narrative Trails (German et al., 2025), que emplea algoritmos de busqueda de caminos de maxima capacidad para identificar secuencias coherentes de documentos. Estos metodos han sido disenados especificamente para modelar relaciones semanticas y temporales entre documentos dentro de una coleccion.

En paralelo, investigaciones recientes han explorado diversos enfoques para la comprension y organizacion de informacion narrativa. Por un lado, se han desarrollado metodos algoritmicos que optimizan la coherencia mediante la compresion de grafos de eventos y transporte optimo (Li et al., 2021). Por otro lado, la emergencia de los modelos de lenguaje de gran escala (LLMs) ha abierto nuevas posibilidades para la construccion de narrativas a partir de colecciones de documentos, gracias a su capacidad de razonamiento semantico.

En el contexto de la extraccion de narrativas basada en eventos, es comun asumir una correspondencia directa entre eventos y documentos, donde cada documento representa un evento dentro de una narrativa (Keith et al., 2023). Bajo este enfoque, la tarea consiste en identificar secuencias documentales que mantengan coherencia temporal y causal dentro de una coleccion.

Los metodos algoritmicos tradicionales, como Narrative Maps y Narrative Trails, han sido disenados especificamente para abordar este problema, modelando explicitamente las relaciones entre documentos mediante tecnicas de optimizacion. Sin embargo, estos enfoques presentan limitaciones en terminos de flexibilidad y capacidad para capturar relaciones semanticas complejas en escenarios altamente dinamicos.

Por otro lado, si bien los modelos de lenguaje han demostrado capacidades avanzadas en tareas de comprension semantica y razonamiento sobre eventos, su uso en extraccion de narrativas basada en eventos a nivel documental se encuentra aun en una etapa exploratoria. La literatura actual no ha explorado de manera sistematica y experimental su aplicacion directa en la extraccion de narrativas basada en eventos a nivel documental. En particular, los modelos de lenguaje propietarios han sido los principales utilizados debido a su alto desempeno, aunque su uso introduce limitaciones asociadas a costos, dependencia tecnologica y restricciones de acceso. Los modelos de lenguaje de codigo abierto

ejecutados localmente han surgido como una alternativa viable, especialmente en escenarios que requieren control sobre los datos y reproducibilidad experimental. Sin embargo, existe una falta de evidencia empirica que permita determinar si los modelos locales pueden alcanzar niveles de coherencia comparables tanto a los metodos algoritmicos tradicionales como a los modelos propietarios.

Esta ausencia de estudios comparativos bajo condiciones controladas limita la comprension del potencial real de los modelos de lenguaje en este dominio, asi como la viabilidad de adoptar soluciones locales como sustitutos de enfoques comerciales o algoritmicos.

1.2. Justificacion

La integracion de modelos de lenguaje en la extraccion de narrativas sugiere que los LLMs podrian ser utilizados no solo para evaluar la calidad de una historia, sino tambien para asistir activamente en su organizacion. La utilizacion de modelos locales responde a la necesidad de evaluar su viabilidad en entornos con restricciones de infraestructura, donde el uso de modelos propietarios mas avanzados no es factible. Se propone una investigacion basica de enfoque cuantitativo orientada a evaluar el desempeno de modelos de lenguaje de codigo abierto ejecutados localmente en tareas de extraccion de narrativas basada en eventos (Keith et al., 2023), realizando una comparacion directa con metodos algoritmicos establecidos en la literatura.

La utilizacion de los baselines Narrative Maps (Keith & Mitra, 2021) y Narrative Trails (German et al., 2025) permite establecer un marco de comparacion controlado entre enfoques disenados especificamente para el problema y modelos de lenguaje que han demostrado capacidades en tareas relacionadas con la organizacion de informacion narrativa.

1.3. Marco teorico

1.3.1. Extraccion de narrativas basada en eventos

La extraccion de narrativas basada en eventos es un area de investigacion que se enfoca en identificar y organizar secuencias de documentos que representan historias coherentes dentro de colecciones documentales. Keith et al. (2023) formalizan este problema asumiendo que cada documento de un conjunto de datos representa un evento dentro de una narrativa. Bajo este supuesto, la tarea consiste en seleccionar subconjuntos de documentos que formen secuencias con coherencia temporal y causal. El trabajo de Shahaf y Guestrin (2010) introdujo la nocion de conectar articulos de noticias para formar cadenas que maximizan la minima coherencia, sentando las bases para los enfoques posteriores.

1.3.2. Modelos de lenguaje de gran escala (LLMs)

Los LLMs son modelos basados en redes neuronales entrenadas sobre grandes volumenes de texto que han demostrado capacidades avanzadas en comprension semantica, razonamiento y generacion de texto. En este trabajo se distingue entre modelos propietarios (e.g., GPT-4o-mini), accesibles via APIs comerciales, y modelos de codigo abierto (e.g., LLaMA, Mistral), ejecutables localmente. Esta distincion es relevante en terminos de costo, reproducibilidad, control sobre los datos y accesibilidad.

1.3.3. Evaluacion LLM-as-a-judge

El protocolo LLM-as-a-judge (Zheng et al., 2023) propone utilizar LLMs como evaluadores automaticos de calidad. Keith (2025) aplica este enfoque para evaluar la coherencia de secuencias documentales, mostrando correlaciones consistentes con metricas de coherencia matematica y juicios humanos. Este protocolo es el metodo de evaluacion principal en la presente investigacion.

1.3.4. Escalabilidad en extraccion narrativa

En el contexto de este trabajo, se considera la escalabilidad en funcion del tamano del conjunto de datos manejable por el metodo en un tiempo acotado (menor o igual a 2 minutos). Bajo estas restricciones, la version base de Narrative Maps maneja bien hasta 1000 documentos aproximadamente en una laptop de gama alta. Adicionalmente, se considera la restriccion de recursos computacionales, considerando que los metodos desarrollados deben ser posibles de ejecutar sin depender de infraestructura costosa. Esta segunda dimension motiva el uso de modelos de lenguaje locales como alternativa a los modelos propietarios mas avanzados.

1.4. Objetivos

1.4.1. Objetivo general

Desarrollar metodos de extraccion de narrativas coherentes mediante el uso de modelos de lenguaje locales, a traves de la comparacion de su desempeno con modelos propietarios y metodos algoritmicos tradicionales.

1.4.2. Objetivos especificos

OE1. Identificar modelos de lenguaje de codigo abierto viables para su ejecucion local y aplicacion en tareas de extraccion de narrativas.

OE2. Implementar un pipeline experimental de extraccion narrativa que integre modelos locales adaptados a hardware limitado.

OE3. Evaluar cuantitativamente la coherencia narrativa y la eficiencia computacional de los modelos implementados.

OE4. Analizar los trade-offs entre calidad narrativa, latencia y costo segun los enfoques evaluados.

1.4.3. Resultados esperados

Tabla 1. Objetivos especificos y resultados esperados.

Objetivo especifico	Resultado esperado
OE1. Identificar modelos de lenguaje de codigo abierto viables	Reporte tecnico con la seleccion y justificacion de los modelos locales evaluados
OE2. Implementar pipeline experimental	Prototipo funcional (pipeline) capaz de extraer narrativas usando modelos locales y baselines
OE3. Evaluar coherencia y eficiencia	Conjunto de datos con resultados cuantitativos de coherencia, tiempos de ejecucion y uso de memoria

Objetivo especifico	Resultado esperado
OE4. Analizar trade-offs calidad/costo	Reporte del analisis comparativo que identifique el modelo local con mejor equilibrio entre calidad y costo

1.5. Metodologia

La investigacion se realiza siguiendo el metodo cientifico bajo un diseno experimental de caracter cuantitativo, orientado a comparar el desempeno de modelos de lenguaje locales en tareas de extraccion de narrativas. La metodologia contempla cinco etapas principales:

1. Revision bibliografica dirigida (OE1): Identificacion de trabajos relevantes en extraccion de narrativas basada en eventos y uso de modelos de lenguaje en organizacion documental. La busqueda se realiza en Google Scholar, ACM Digital Library, IEEE Xplore y ACL Anthology.
2. Seleccion de modelos y definicion experimental (OE1 + OE2): Seleccion de modelos de lenguaje de codigo abierto ejecutables localmente, priorizando compatibilidad con hardware limitado. Se evaluan modelos disponibles a traves de Ollama.
3. Implementacion de metodos y ejecucion de experimentos (OE2): Implementacion de los metodos algoritmicos (Narrative Maps y Narrative Trails) y diseno de prompts para los modelos de lenguaje. Cada metodo se ejecuta multiples veces para asegurar estabilidad en los resultados.
4. Evaluacion y analisis de resultados (OE3): Evaluacion mediante metricas cuantitativas de coherencia narrativa (avg_edge_coherence, judge score) y eficiencia computacional (tiempo de ejecucion, uso de memoria). Validacion estadistica mediante test-t de Welch con correccion de Bonferroni y calculo de Cohen's d.
5. Sintesis de resultados (OE4): Analisis comparativo para identificar los trade-offs entre calidad narrativa y eficiencia computacional.

1.6. Contenido del documento

El primer capitulo de este informe aborda la introduccion del proyecto, en el cual se describen los aspectos generales incluyendo el contexto en el que se desarrolla, la problematica identificada, la justificacion de su abordaje, el marco teorico, los objetivos y la metodologia, el segundo capitulo presenta la revision bibliografica realizada y el proceso de seleccion de los modelos de lenguaje locales utilizados en la investigacion, correspondiente al cumplimiento del primer objetivo especifico.

El tercer capitulo detalla el diseno e implementacion del pipeline experimental, incluyendo la arquitectura del sistema, los metodos de scoring, los algoritmos de extraccion y el protocolo de evaluacion, correspondiente al segundo objetivo especifico.

El cuarto capitulo presenta los experimentos ejecutados y los resultados preliminares obtenidos, incluyendo el analisis estadistico de las comparaciones entre metodos, correspondiente al avance en los objetivos especificos tercero y cuarto y finalmente el quinto capitulo presenta las discusiones

preliminares derivadas del avance del proyecto, el estado de cumplimiento de los objetivos y el trabajo restante.

El Anexo A contiene la estructura detallada del repositorio del proyecto, una descripción del notebook utilizado para la ejecución en Google Colab y el formato de los archivos de resultados generados.

CAPITULO II: REVISION BIBLIOGRAFICA Y SELECCION DE MODELOS

El presente capitulo describe el proceso de revision bibliografica realizado y la seleccion de los modelos de lenguaje locales utilizados en la investigacion. Estas actividades corresponden al cumplimiento del primer objetivo especifico (OE1).

2.1. Revision bibliografica

La revision bibliografica se realizo durante las primeras tres semanas del proyecto (actividades 1 a 3 de la planificacion), siguiendo un protocolo de revision dirigida orientado a identificar trabajos relevantes en dos lineas tematicas principales: la extraccion de narrativas basada en eventos y el uso de modelos de lenguaje en tareas de organizacion documental. La estrategia de busqueda contemplo cinco fuentes academicas complementarias: Google Scholar como buscador general de cobertura amplia, ACM Digital Library para literatura en ciencias de la computacion e interaccion humano-computador, IEEE Xplore para ingenieria e inteligencia artificial, ACL Anthology para procesamiento de lenguaje natural, y los servicios bibliograficos de la Universidad Catolica del Norte para el acceso a recursos suscritos a traves de la institucion.

Las consultas de busqueda se construyeron combinando terminos clave en ingles, dado que la mayor parte de la literatura relevante en este dominio se publica en ese idioma. Las queries principales utilizadas fueron: "narrative extraction", "event-based narrative extraction", "narrative maps", "narrative trails", "story chain extraction", "connecting the dots news", "timeline summarization", "narrative coherence evaluation", "LLM-as-a-judge", "local language models" y "open-source LLMs", junto con combinaciones de estos terminos mediante operadores booleanos (AND, OR) y filtros por anio de publicacion (preferentemente 2020-2025) para priorizar literatura reciente sin descartar los trabajos fundacionales del campo. Adicionalmente, se realizo una busqueda por citas hacia adelante y hacia atras (forward y backward snowballing) a partir de los trabajos seminales identificados (Shahaf y Guestrin, 2010; Keith y Mitra, 2021), con el objetivo de cubrir tanto los antecedentes historicos como los desarrollos mas recientes derivados de estos enfoques.

Los criterios de inclusion contemplaron: trabajos publicados en conferencias o revistas con revision por pares, trabajos accesibles en texto completo, trabajos en ingles o espanol, y trabajos cuyo contenido se relacionara directamente con la extraccion de narrativas, la evaluacion de coherencia textual o la aplicacion de modelos de lenguaje a tareas documentales. Se excluyeron trabajos centrados exclusivamente en generacion de texto creativo sin componente de extraccion, resumen extractivo clasico sin enfasis en coherencia narrativa, y aplicaciones a dominios altamente especificos sin generalizacion al caso documental abierto.

2.1.1. Extraccion de narrativas basada en eventos

El trabajo fundacional de Shahaf y Guestrin (2010) introdujo el concepto de conectar articulos de noticias para formar cadenas narrativas coherentes, formulando el problema como la busqueda de

caminos que maximizan la mínima coherencia entre documentos consecutivos. Keith et al. (2023) presentan una revisión sistemática del campo, formalizando la extracción de narrativas basada en eventos como la tarea de seleccionar subconjuntos de documentos que forman secuencias temporal y causalmente coherentes.

En cuanto a métodos específicos, Narrative Maps (Keith & Mitra, 2021) formula la selección de documentos como un problema de programación lineal entera y lo resuelve mediante una relajación de programación lineal fraccional (LP relaxation). Esta aproximación modela la selección de documentos como un problema de optimización que maximiza la coherencia de la narrativa, utilizando variables continuas en lugar de binarias. El método utiliza restricciones de flujo para garantizar la conectividad del subgrafo resultante. Narrative Trails (German et al., 2025) propone un enfoque alternativo basado en el algoritmo MaxiMin Dijkstra, que busca el camino de n nodos que maximiza la mínima coherencia entre aristas consecutivas (bottleneck path). Este enfoque privilegia la consistencia local sobre la optimización global.

Adicionalmente, Li et al. (2021) proponen la generación de líneas de tiempo mediante compresión de grafos de eventos con transporte óptimo, un enfoque relacionado pero orientado a la summarización temporal más que a la extracción narrativa.

2.1.2. Uso de LLMs en tareas narrativas

La emergencia de los modelos de lenguaje de gran escala basados en la arquitectura Transformer (Vaswani et al., 2017) ha transformado el campo del procesamiento de lenguaje natural durante los últimos años. Trabajos como BERT (Devlin et al., 2019) y GPT-3 (Brown et al., 2020) demostraron que los modelos preentrenados sobre grandes corpus de texto pueden alcanzar desempeños competitivos en múltiples tareas de comprensión semántica con un esfuerzo de ajuste mínimo. Más recientemente, la disponibilidad de modelos de código abierto como LLaMA (Touvron et al., 2023) y Mistral 7B (Jiang et al., 2023) ha permitido la ejecución local de modelos competitivos sin depender de servicios comerciales, abriendo oportunidades para aplicaciones en entornos con restricciones de costo, privacidad o conectividad.

En el ámbito específico de la representación semántica de documentos, los modelos de embeddings derivados de Sentence-BERT (Reimers y Gurevych, 2019) han establecido un estándar de facto para el cálculo de similitud semántica entre fragmentos de texto, sirviendo como baseline en numerosas aplicaciones de recuperación de información y agrupamiento documental. En el presente trabajo, el modelo all-MiniLM-L6-v2 de la biblioteca Sentence-Transformers se utiliza como scorer algorítmico de referencia.

La literatura reciente muestra un creciente interés en el uso de modelos de lenguaje para tareas de organización y evaluación de información narrativa. Sin embargo, la aplicación directa de LLMs como scorers de coherencia entre pares de documentos en el contexto de extracción narrativa basada en eventos no ha sido explorada sistemáticamente, ni se han realizado comparaciones empíricas que enfrenten modelos locales de código abierto contra alternativas comerciales y baselines algorítmicos bajo condiciones experimentales idénticas. Esta brecha constituye una de las motivaciones centrales de la presente investigación.

2.1.3. Evaluación automática con LLM-as-a-judge

El protocolo LLM-as-a-judge (Zheng et al., 2023) propone utilizar modelos de lenguaje como evaluadores automáticos, ofreciendo una alternativa escalable a la evaluación humana. Keith (2025) valida este enfoque específicamente para la evaluación de coherencia narrativa, demostrando correlaciones consistentes con métricas matemáticas de coherencia y con juicios humanos. En la presente investigación se adopta este protocolo como método de evaluación principal, utilizando tanto evaluación pointwise (puntuación en escala 0-10) como pairwise (comparación directa entre dos narrativas).

2.2. Selección de modelos de lenguaje locales

La selección de modelos (actividad 4) se realizó considerando los siguientes criterios: disponibilidad pública y licencia de código abierto, tamaño del modelo compatible con hardware limitado (Apple M4 Pro, 18 GB RAM, sin GPU dedicada), capacidad de inferencia local a través de Ollama, y desempeño reportado en tareas de comprensión semántica.

Se evaluaron inicialmente cuatro modelos candidatos:

Tabla 2. Modelos candidatos evaluados.

Modelo	Tamaño	Estado	Observación
LLaMA 3.2	2.0 GB	Seleccionado	Rápido, buen rendimiento, bajo consumo de recursos
Mistral 7B	4.4 GB	Seleccionado	Mayor capacidad semántica, tiempo de inferencia moderado
Phi-3	2.3 GB	Descartado	Sobrecalentamiento del hardware durante ejecución prolongada
Gemma 2	5.4 GB	Descartado	Sobrecalentamiento del hardware y consumo excesivo de memoria

Los modelos Phi-3 y Gemma 2 fueron descartados tras pruebas iniciales que revelaron problemas de sobrecalentamiento en el equipo de desarrollo, lo que impedía la ejecución estable de los experimentos. Los modelos LLaMA 3.2 y Mistral 7B demostraron ser ejecutables de forma estable en sesiones prolongadas.

Adicionalmente, se incluyó el modelo propietario GPT-4o-mini (OpenAI) como punto de referencia superior, accesible vía API. Como baseline algorítmico se utilizó el modelo de embeddings all-MiniLM-L6-v2 de la biblioteca Sentence-Transformers, que calcula coherencia mediante similitud coseno entre representaciones vectoriales de documentos.

CAPITULO III: DISEÑO E IMPLEMENTACIÓN DEL PIPELINE EXPERIMENTAL

El presente capítulo describe el diseño y la implementación del pipeline experimental de extracción narrativa, correspondiente al cumplimiento del segundo objetivo específico (OE2). Se detallan la arquitectura del sistema, los componentes implementados, el diseño experimental y el dataset utilizado.

3.1. Arquitectura del pipeline

El pipeline fue implementado en Python y sigue una arquitectura modular compuesta por cinco componentes principales: carga de datos, scoring de coherencia, extracción narrativa, evaluación y registro de métricas.

La Figura 3.1 muestra el flujo general del pipeline. Primero, se carga el dataset en formato JSONL y se ordenan los documentos por fecha de publicación. Luego, se calcula una matriz de coherencia entre todos los pares de documentos utilizando uno de los scorers disponibles. A continuación, se aplica uno de los algoritmos de extracción (Narrative Maps o Narrative Trails) para seleccionar la secuencia de n documentos más coherente. Finalmente, se evalúa la narrativa resultante mediante el protocolo LLM-as-a-judge y se registran las métricas de rendimiento.

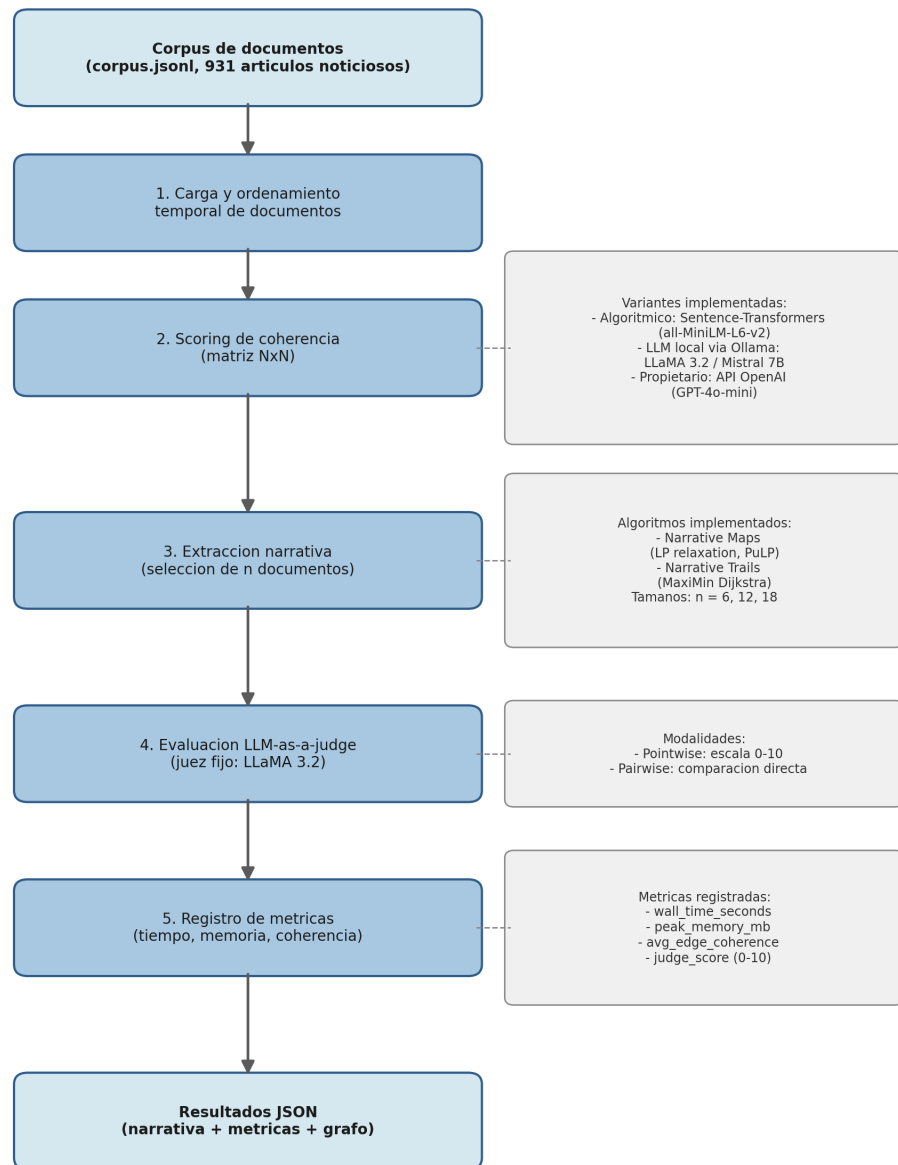


Figura 1. Arquitectura general del pipeline experimental.

3.2. Componentes del sistema

Esta sección describe la implementación de cada componente con el detalle necesario para comprender su rol en el pipeline. La organización completa de los archivos del repositorio, incluyendo la ubicación exacta de cada módulo dentro de la estructura de directorios, se presenta en el Anexo A.1.

3.2.1. Módulo de scoring

El modulo de scoring es responsable de calcular la puntuacion de coherencia entre pares de documentos. Se implementaron tres variantes:

- Scorer algoritmico (scoring/algorithmic.py): Utiliza el modelo all-MiniLM-L6-v2 de Sentence-Transformers para generar embeddings de cada documento y calcula la similitud coseno entre pares. Este enfoque sirve como baseline al representar el estado del arte en metodos no basados en LLMs.
- Scorer LLM local (llm/scorer.py): Envia pares de documentos a un modelo de lenguaje local ejecutado via Ollama. El prompt solicita al modelo evaluar la coherencia narrativa entre los documentos en una escala de 0 a 1. Se implementaron variantes para LLaMA 3.2 y Mistral 7B.
- Scorer propietario (llm/openai_scorer.py): Utiliza el mismo protocolo que el scorer LLM local, pero invocando la API de OpenAI con el modelo GPT-4o-mini. Permite la comparacion directa entre modelos locales y propietarios bajo condiciones identicas.

Para optimizar el rendimiento, se implemento un sistema de precomputo y cache de matrices de coherencia (scoring/precompute.py), evitando recalculos innecesarios entre repeticiones del mismo experimento.

3.2.2. Modulo de extraccion

Se implementaron dos algoritmos de extraccion:

- Narrative Maps (baselines/maps.py): Implementacion basada en Keith y Mitra (2021). Formula el problema como programacion lineal entera y lo resuelve mediante una relajacion LP fraccional utilizando la biblioteca PuLP, seleccionando un subgrafo de n documentos que maximiza la minima coherencia entre aristas (maximin). Las variables de decision son continuas en $[0, 1]$ y se aplican restricciones de flujo para garantizar conectividad. El algoritmo retorna no solo el camino principal, sino toda la estructura de grafo con nodos seleccionados y aristas con sus respectivas puntuaciones de coherencia.
- Narrative Trails (baselines/trails.py): Implementacion basada en German et al. (2025). Utiliza una variante del algoritmo de Dijkstra (MaxiMin) para encontrar el camino de n nodos que maximiza la minima coherencia entre aristas consecutivas. A diferencia de Narrative Maps, este metodo produce un camino lineal en lugar de un subgrafo.

3.2.3. Modulo de evaluacion

La evaluacion (evaluation/judge.py) implementa el protocolo LLM-as-a-judge en dos modalidades: Evaluacion pointwise: Un modelo de lenguaje evalua la coherencia de una narrativa individual en una escala de 0 a 10, proporcionando una justificacion textual.

Evaluacion pairwise: Comparacion directa entre dos narrativas, donde el modelo indica cual presenta mayor coherencia.

Se utilizo LLaMA 3.2 como modelo juez fijo para todos los experimentos, asegurando consistencia en las evaluaciones.

3.2.4. Registro de metricas

El modulo de metricas (metrics/recorder.py) registra automaticamente para cada experimento: el tiempo total de ejecucion (wall_time_seconds), el uso maximo de memoria (peak_memory_mb) y la coherencia promedio entre aristas consecutivas (avg_edge_coherence).

3.3. Dataset

3.3.1. Corpus completo

El dataset base corresponde a una coleccion de 931 articulos noticiosos chilenos almacenados en FINAL_DATA.json, obtenidos de diversos medios de comunicacion (incluyendo Ciper Chile) mediante tecnicas de web scraping. Los articulos cubren multiples temas de la agenda noticiosa chilena.

3.3.2. Dataset experimental

Los experimentos se ejecutaron sobre el corpus completo (corpus.jsonl, 931 articulos). Para manejar el volumen de datos, se utilizo una ventana temporal de 30 dias y un limite de 200 documentos por ventana. Se definieron cuatro puntos de inicio en el corpus (indices 0, 50, 100 y 150), lo que permite evaluar la robustez de los metodos sobre distintos segmentos tematicos y temporales de la coleccion. De este modo, cada metodo opera sobre diferentes subconjuntos del corpus, evitando sesgos derivados de un unico segmento tematico.

3.4. Diseno experimental

3.4.1. Combinaciones evaluadas

Cada combinacion de algoritmo de extraccion y scorer de coherencia produce un metodo. Se evaluaron ocho metodos en total, como se muestra en la Tabla 3.

Tabla 3. Metodos evaluados.

Metodo	Algoritmo	Scorer	Modelo
maps_algorithmic	Narrative Maps (LP relaxation)	Sentence-Transformers	all-MiniLM-L6-v2
maps_llm (llama3.2)	Narrative Maps (LP relaxation)	LLM local (Ollama)	LLaMA 3.2
maps_llm (mistral)	Narrative Maps (LP relaxation)	LLM local (Ollama)	Mistral 7B
maps_proprietary	Narrative Maps (LP relaxation)	Propietario (API)	GPT-4o-mini
trails_algorithmic	Narrative Trails (MaxiMin)	Sentence-Transformers	all-MiniLM-L6-v2
trails_llm (llama3.2)	Narrative Trails (MaxiMin)	LLM local (Ollama)	LLaMA 3.2

Metodo	Algoritmo	Scorer	Modelo
trails_llm (mistral)	Narrative Trails (MaxiMin)	LLM local (Ollama)	Mistral 7B
trails_proprietary	Narrative Trails (MaxiMin)	Propietario (API)	GPT-4o-mini

3.4.2. Parametros experimentales

Tabla 4. Parametros experimentales.

Parametro	Valor
Tamanos de narrativa (n)	6, 12 y 18 documentos
Ventana temporal	30 días
Maximo de documentos por ventana	200
Puntos de inicio en el corpus	0, 50, 100 y 150
Repeticiones por combinacion (LLM)	5
Repeticiones por combinacion (algoritmico)	1 (determinista)
Modelo juez (LLM-as-a-judge)	LLaMA 3.2 (fijo para todos)
Total de experimentos	384

Se definieron tres tamanos de narrativa ($n=6$, $n=12$ y $n=18$) para analizar el comportamiento de los metodos bajo diferentes niveles de complejidad. Los cuatro puntos de inicio permiten evaluar la robustez de los resultados sobre distintos segmentos del corpus. Los metodos basados en LLM se ejecutaron 5 veces por configuracion para capturar variabilidad, mientras que el metodo algoritmico se ejecuto 1 vez dado su caracter determinista.

3.4.3. Metricas

Tabla 5. Metricas registradas.

Metrica	Descripcion
avg_edge_coherence	Promedio de coherencia entre pares consecutivos de documentos en la narrativa. Rango [0, 1].
wall_time_seconds	Tiempo total de ejecucion del metodo (scoring + extraccion).
peak_memory_mb	Uso maximo de memoria durante la ejecucion.

Metrica	Descripcion
judge_score	Puntuacion de coherencia (0-10) asignada por el LLM evaluador.

3.4.4. Evaluacion estadistica

Para la comparacion entre metodos se utilizaron las siguientes herramientas estadisticas:

- Test-t de Welch (varianzas no iguales) para comparaciones entre pares de metodos.
- Correccion de Bonferroni para ajustar el nivel de significancia ante comparaciones multiples.
- Cohen's d para cuantificar el tamano del efecto.
- Nivel de significancia: $\alpha = 0.05$.

3.5. Infraestructura

Tabla 6. Infraestructura utilizada.

Componente	Detalle
Hardware (desarrollo)	Apple M4 Pro, 18 GB RAM, sin GPU dedicada
Hardware (ejecucion)	Google Colab, GPU NVIDIA A100
Lenguaje	Python 3.14
Framework LLM local	Ollama 0.21.1
Paralelismo	8 workers simultaneos
Dependencias principales	sentence-transformers, networkx, PuLP, ollama, openai, scipy, pandas, matplotlib

La ejecucion de los 384 experimentos sobre el corpus completo se realizo en Google Colab con GPU A100, dado que el volumen de experimentos y el tamano del corpus excedian la capacidad del hardware de desarrollo local. El paralelismo de 8 workers permitio completar la ejecucion en aproximadamente 1 hora. Los detalles del notebook utilizado para orquestar esta ejecucion en Colab, incluyendo la consolidacion del codigo del repositorio en celdas, el sistema de caching perezoso y el flujo de subida del corpus y descarga de resultados, se describen en el Anexo A.2.

CAPITULO IV: EXPERIMENTOS Y RESULTADOS PRELIMINARES

El presente capitulo presenta los experimentos ejecutados y los resultados obtenidos, correspondientes a los objetivos especificos tercero y cuarto (OE3 y OE4).

4.1. Ejecucion de experimentos

Se ejecutaron 384 experimentos en total sobre el corpus completo (931 articulos), cubriendo las ocho combinaciones de metodo, tres tamanos de narrativa (n=6, 12, 18), cuatro puntos de inicio en el corpus (indices 0, 50, 100, 150) y cinco repeticiones por configuracion para los metodos basados en LLM (una repeticion para los algoritmos, dado su caracter determinista). La ejecucion se realizo en Google Colab con GPU A100, utilizando paralelismo de 8 workers simultaneos para optimizar el tiempo total de ejecucion. Cada experimento genera un archivo JSON con la narrativa extraida, las metricas de coherencia y eficiencia, y la evaluacion del juez LLM; el formato detallado de estos archivos se presenta en el Anexo A.3.

4.2. Resultados de coherencia

4.2.1. Estadisticas descriptivas

La Tabla 7 presenta las estadisticas descriptivas de coherencia promedio (avg_edge_coherence) para cada metodo, agregando los resultados de los tres tamanos de narrativa y los cuatro puntos de inicio.

Tabla 7. Ranking de metodos por coherencia promedio.

#	Metodo	Coherencia media	Desv. est.	Observaciones
1	Trails (Algoritmico)	0.694	0.025	12
2	Trails (GPT-4o-mini)	0.649	0.146	60
3	Maps (Algoritmico)	0.581	0.042	12
4	Maps (GPT-4o-mini)	0.516	0.077	60
5	Trails (LLaMA 3.2)	0.500	0.202	60
6	Trails (Mistral)	0.347	0.245	60
7	Maps (LLaMA 3.2)	0.339	0.062	60
8	Maps (Mistral)	0.223	0.176	60

Los resultados sobre el corpus completo revelan una jerarquia distinta a la observada en los experimentos preliminares sobre el dataset curado: los metodos algoritmos basados en embeddings obtienen la mayor coherencia y la menor variabilidad, seguidos por el modelo propietario GPT-4o-mini, y finalmente los modelos locales. Dentro de los LLMs locales, LLaMA 3.2 supera a Mistral en ambos algoritmos de extraccion.

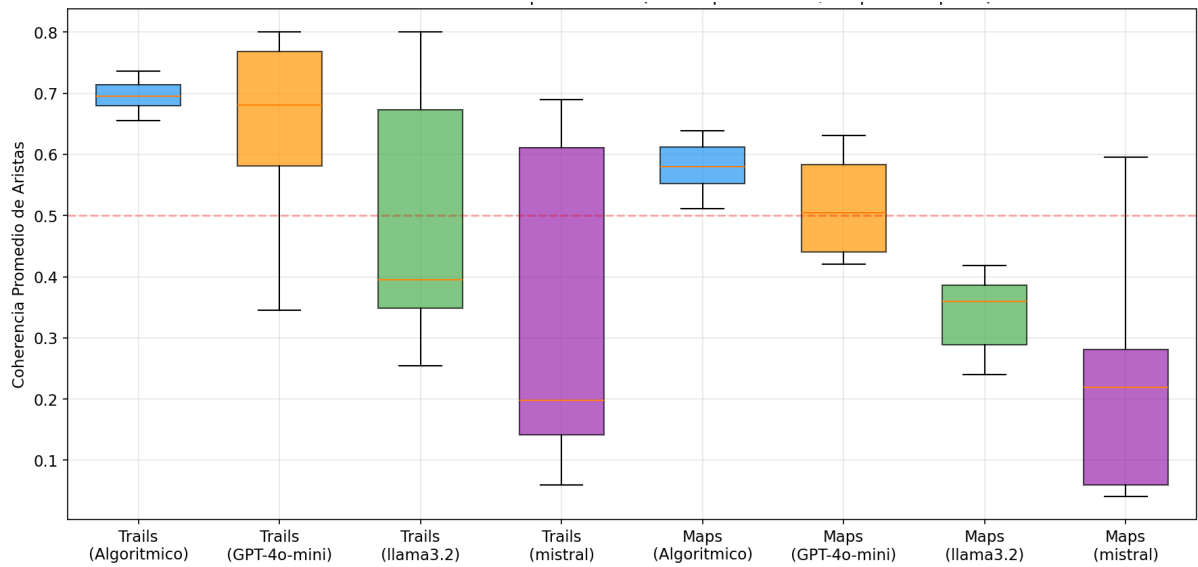


Figura 2. Box plot de coherencia promedio por metodo (384 experimentos).

4.2.2. Efecto del tamaño de narrativa

La Tabla 8 presenta la coherencia desglosada por tamaño de narrativa para cada metodo.

Tabla 8. Coherencia promedio por metodo y tamaño de narrativa.

Metodo	n=6	n=12	n=18
Trails (Algoritmico)	0.714	0.688	0.681
Trails (GPT-4o-mini)	0.745	0.605	0.599
Maps (Algoritmico)	0.576	0.570	0.597
Maps (GPT-4o-mini)	0.470	0.536	0.542
Trails (LLaMA 3.2)	0.590	0.507	0.403
Trails (Mistral)	0.495	0.298	0.249
Maps (LLaMA 3.2)	0.340	0.345	0.330
Maps (Mistral)	0.195	0.256	0.218

Se observan dos patrones diferenciados segun el tipo de scorer:

- Los metodos con scorer LLM (local y propietario) muestran una degradacion de coherencia al aumentar el tamaño de la narrativa, especialmente pronunciada en los LLMs locales. Por ejemplo, Trails (LLaMA 3.2) cae de 0.590 (n=6) a 0.403 (n=18), una reduccion del 32%.
- Los metodos algoritmicos muestran estabilidad notable: Trails (Algoritmico) solo cae de 0.714 a 0.681 (reduccion del 5%), y Maps (Algoritmico) incluso mejora ligeramente de 0.576 a 0.597.

Esta diferencia se explica porque el scorer algoritmico (embeddings) produce matrices de coherencia densas y estables, mientras que los scorers LLM generan matrices mas dispersas, lo que dificulta la extraccion de narrativas largas coherentes.

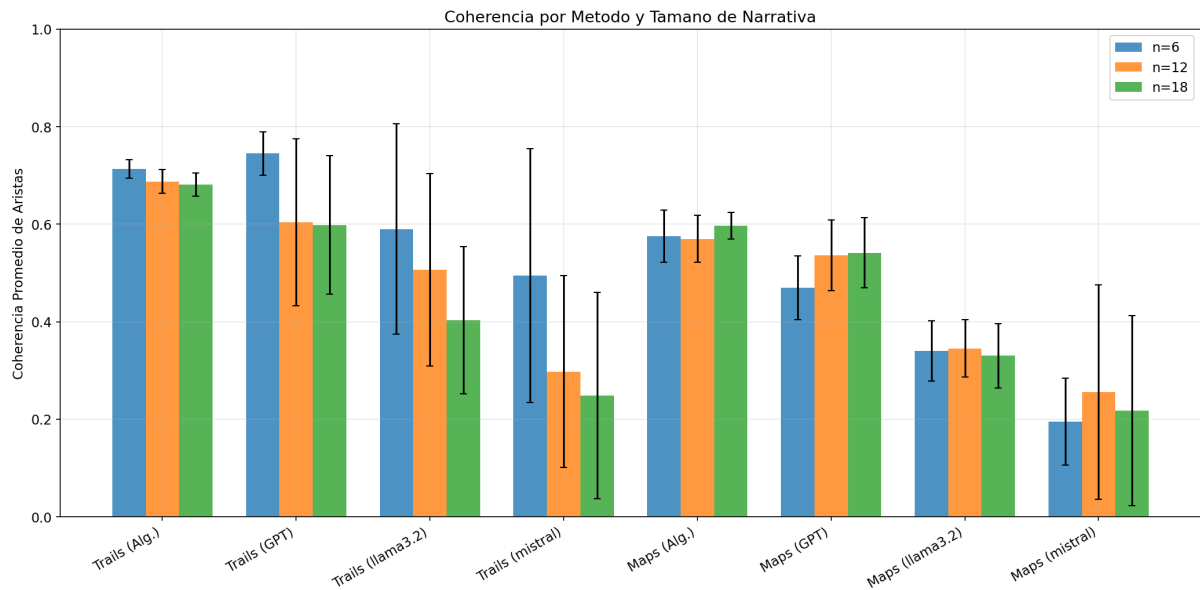


Figura 3. Coherencia por metodo y tamano de narrativa (n=6, n=12, n=18).

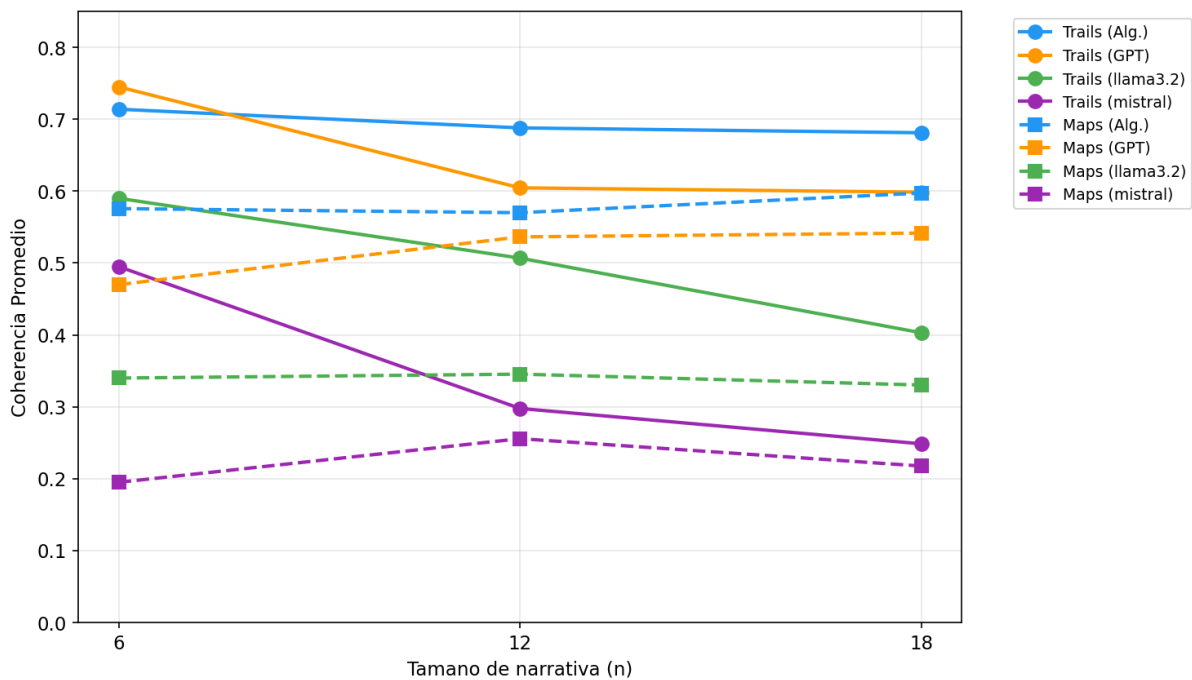


Figura 4. Efecto del tamano de narrativa en la coherencia.

4.2.3. Narrative Trails vs. Narrative Maps

Para un mismo scorer, Narrative Trails supera consistentemente a Narrative Maps en coherencia de aristas. La Tabla IV-3 resume esta comparacion.

Table 9. Comparacion Trails vs. Maps por scorer (coherencia media global).

Scorer	Trails	Maps	Diferencia
Algoritmico	0.694	0.581	+0.113
GPT-4o-mini	0.649	0.516	+0.133
LLaMA 3.2	0.500	0.339	+0.161
Mistral	0.347	0.223	+0.124

La ventaja de Trails es consistente en todos los tamanos de narrativa y todos los scorers. Esto se explica por la naturaleza del algoritmo MaxiMin de Trails, que maximiza la minima coherencia entre aristas consecutivas, produciendo caminos con coherencia mas uniforme. Maps, al maximizar la coherencia total del subgrafo, puede incluir aristas de baja coherencia que reducen el promedio.

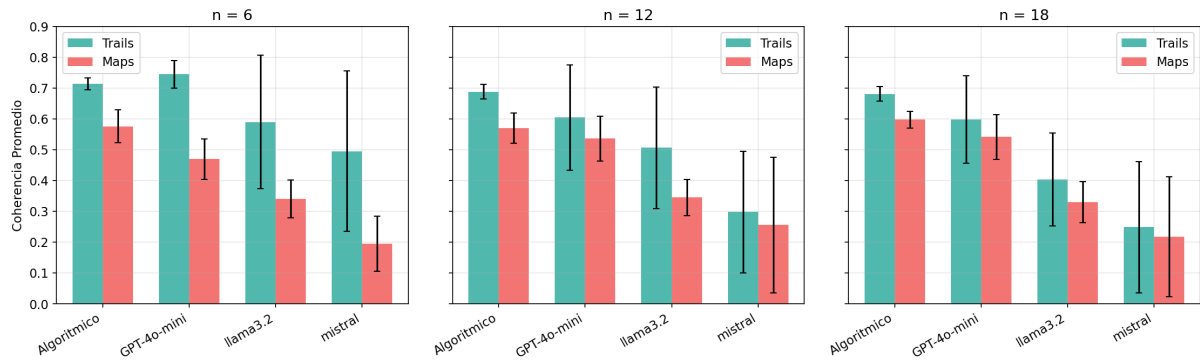


Figura 5. . Trails vs. Maps por scorer y tamano de narrativa.

4.2.4. Robustez entre segmentos del corpus

Se analizo la estabilidad de los metodos a traves de los cuatro puntos de inicio en el corpus (indices 0, 50, 100, 150), que representan diferentes segmentos tematicos de la coleccion de noticias.

Los metodos algoritmicos muestran alta estabilidad entre segmentos (desviacion estandar < 0.05). En contraste, los LLMs locales presentan alta variabilidad: Trails (Mistral) oscila entre 0.11 y 0.63 dependiendo del segmento, lo que indica sensibilidad al contenido tematico del corpus.

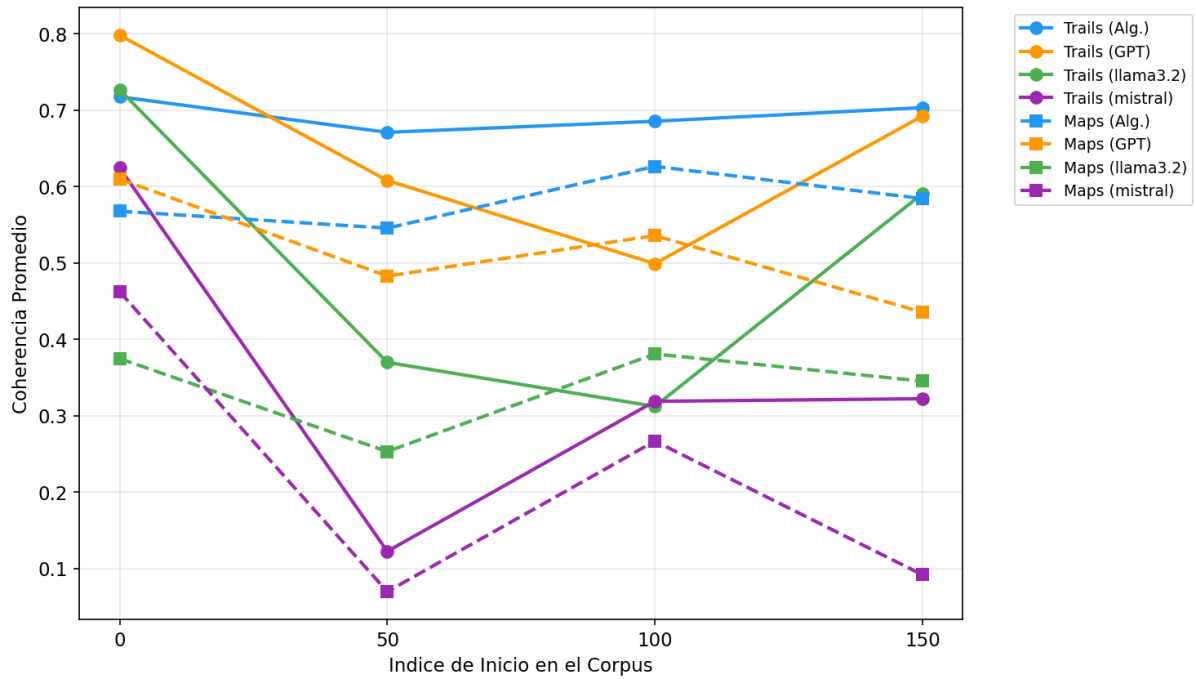


Figura 6. Robustez: Coherencia por punto de inicio en el corpus.

4.3. Comparaciones estadísticas

Para validar formalmente las diferencias observadas en las estadísticas descriptivas, se realizaron comparaciones por pares entre todos los métodos utilizando el test-t de Welch (que no asume igualdad de varianzas, condición relevante dado que la dispersión de los LLMs locales es notoriamente mayor que la de los métodos algorítmicos). Para controlar el error tipo I derivado de la realización de múltiples comparaciones simultáneas, se aplicó la corrección de Bonferroni sobre los p-valores. Adicionalmente, se calculó el tamaño del efecto mediante Cohen's d, que cuantifica la magnitud de la diferencia entre medias en unidades de desviación estándar y permite distinguir entre diferencias estadísticamente significativas pero prácticamente irrelevantes y diferencias con impacto sustantivo. Los criterios convencionales interpretan $|d| \sim 0.2$ como efecto pequeño, $|d| \sim 0.5$ como mediano y $|d| \geq 0.8$ como grande.

La Tabla 10 presenta las comparaciones estadísticamente significativas más relevantes para la coherencia.

Tabla 10. Comparaciones significativas en coherencia ($p_{\text{bonferroni}} < 0.05$).

Comparacion		Coh. A	Coh. B	p (Bonf.)	Cohen's d	Interpretacion
maps_algorithmic vs. maps_llm		0.581	0.281	< 0.001	2.18	Algoritmico muy superior a LLM local
maps_algorithmic vs. maps_proprietary		0.581	0.516	< 0.001	0.90	Algoritmico superior a propietario

Comparacion		Coh. A	Coh. B	p (Bonf.)	Cohen's d	Interpretacion
maps_algorithmic trails_algorithmic	vs.	0.581	0.694	< 0.001	-3.28	Trails superior a Maps
maps_llm maps_proprietary	vs.	0.281	0.516	< 0.001	-1.88	Propietario muy superior a LLM local
trails_algorithmic trails_llm	vs.	0.694	0.424	< 0.001	1.20	Algoritmico superior a LLM local
trails_llm trails_proprietary	vs.	0.424	0.649	< 0.001	-1.07	Propietario superior a LLM local
maps_llm vs. trails_llm		0.281	0.424	< 0.001	-0.73	Trails superior a Maps con LLM

Los resultados confirman estadisticamente las tendencias observadas en el ranking descriptivo y revelan tres patrones principales. En primer lugar, las comparaciones entre el scorer algoritmico y los LLMs locales muestran tamanos de efecto muy grandes (Cohen's d de 2.18 para Maps y 1.20 para Trails), lo que indica que la superioridad de los metodos basados en embeddings no es marginal, sino que excede ampliamente el umbral convencional de efecto grande. En segundo lugar, la comparacion entre Narrative Maps y Narrative Trails con scorer algoritmico arroja el mayor tamano de efecto observado (Cohen's d = -3.28), confirmando que la eleccion del algoritmo de extraccion tiene un impacto mas significativo sobre la coherencia que la eleccion del scorer dentro de la categoria algoritmica. En tercer lugar, las comparaciones que involucran al modelo propietario muestran que GPT-4o-mini supera de forma significativa a los LLMs locales (d = -1.88 para Maps y d = -1.07 para Trails), lo que evidencia una brecha sustantiva entre los modelos de codigo abierto evaluados y la alternativa comercial bajo condiciones experimentales identicas.

Si bien la mayoría de las comparaciones por pares resultaron estadisticamente significativas tras la correccion de Bonferroni, se identificaron dos casos en los que la diferencia entre metodos no alcanza significancia, presentados en la Tabla IV-5.

La Tabla 11 presenta las comparaciones sin diferencias significativas.

Tabla 11. Comparaciones no significativas ($p_{\text{bonferroni}} \geq 0.05$).

Comparacion		p (Bonferroni)	Interpretacion
maps_algorithmic trails_proprietary	vs.	0.050	Algoritmico comparable a propietario con otro algoritmo
trails_algorithmic trails_proprietary	vs.	0.432	Algoritmico y propietario comparables en Trails

Estos resultados son particularmente relevantes desde una perspectiva practica. La ausencia de diferencia significativa entre Trails con scorer algoritmico y Trails con GPT-4o-mini ($p_{\text{bonferroni}} = 0.432$) indica que ambos enfoques son estadisticamente equivalentes en terminos de coherencia, lo que sugiere que el modelo propietario alcanza un desempeno comparable al baseline algoritmico cuando se combina con el algoritmo de extraccion adecuado. Por su parte, la comparacion entre Maps algoritmico y Trails propietario se ubica justo en el limite de la significancia ($p_{\text{bonferroni}} = 0.050$), reforzando la idea de que la combinacion de algoritmo y scorer determina la calidad final mas que el tipo de scorer aisladamente. En sintesis, los resultados estadisticos respaldan la conclusion de que el scorer algoritmico basado en embeddings, en combinacion con Narrative Trails, ofrece el mejor desempeno de coherencia entre los metodos evaluados, mientras que los LLMs locales se posicionan consistentemente como la opcion de menor calidad relativa.

Para visualizar de manera integral la magnitud de las diferencias entre todos los pares de metodos, la Figura 4.6 presenta un heatmap del tamano de efecto (Cohen's d) que permite identificar rapidamente las comparaciones con efectos grandes (celdas mas saturadas) frente a aquellas con diferencias menores (celdas mas tenues).

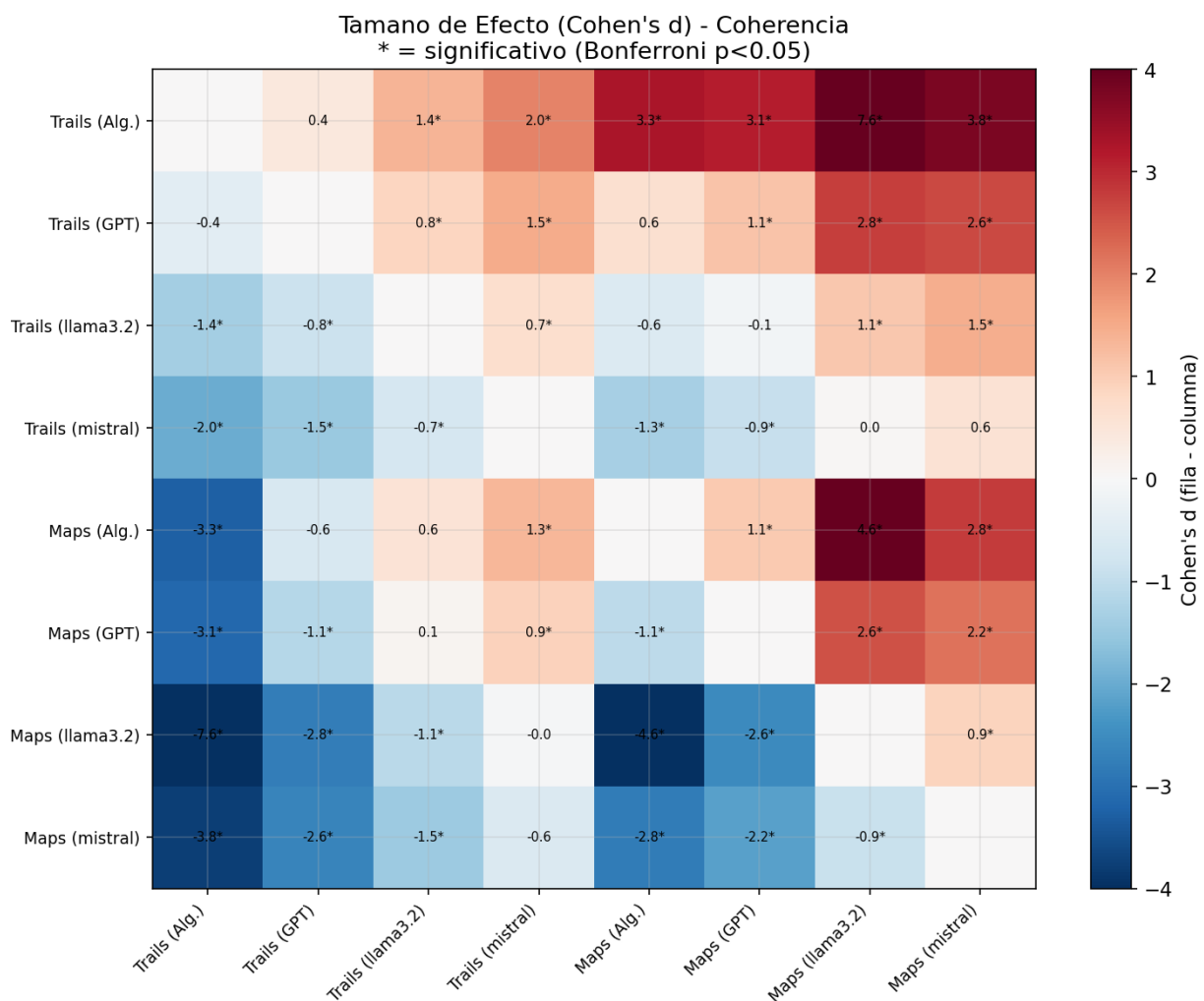


Figura 7. Heatmap de tamano de efecto (Cohen's d) entre pares de metodos.

4.4. Evaluacion LLM-as-a-judge

La Tabla 12 presenta los judge scores promedio por metodo.

Tabla 12. Judge scores promedio por metodo.

Metodo	Judge score medio	Desv. est.
Maps (GPT-4o-mini)	8.0	0.0
Maps (Algoritmico)	7.8	0.4
Maps (LLaMA 3.2)	7.8	0.6
Maps (Mistral)	8.0	0.0
Trails (Algoritmico)	7.7	0.5
Trails (GPT-4o-mini)	7.5	0.9
Trails (Mistral)	7.8	0.6
Trails (LLaMA 3.2)	7.3	0.9

Los judge scores se concentran en el rango 7-8 para todos los metodos, con diferencias minimas entre ellos. Este resultado contrasta marcadamente con las diferencias observadas en `avg_edge_coherence`, donde los metodos algoritmicos superan claramente a los LLMs locales. La divergencia sugiere que el juez LLM evalua aspectos holísticos de la narrativa (tematica, progresion general) que no se capturan en la metrica de coherencia entre pares de aristas, o bien que el juez LLaMA 3.2 tiene limitaciones para discriminar entre niveles de calidad narrativa.

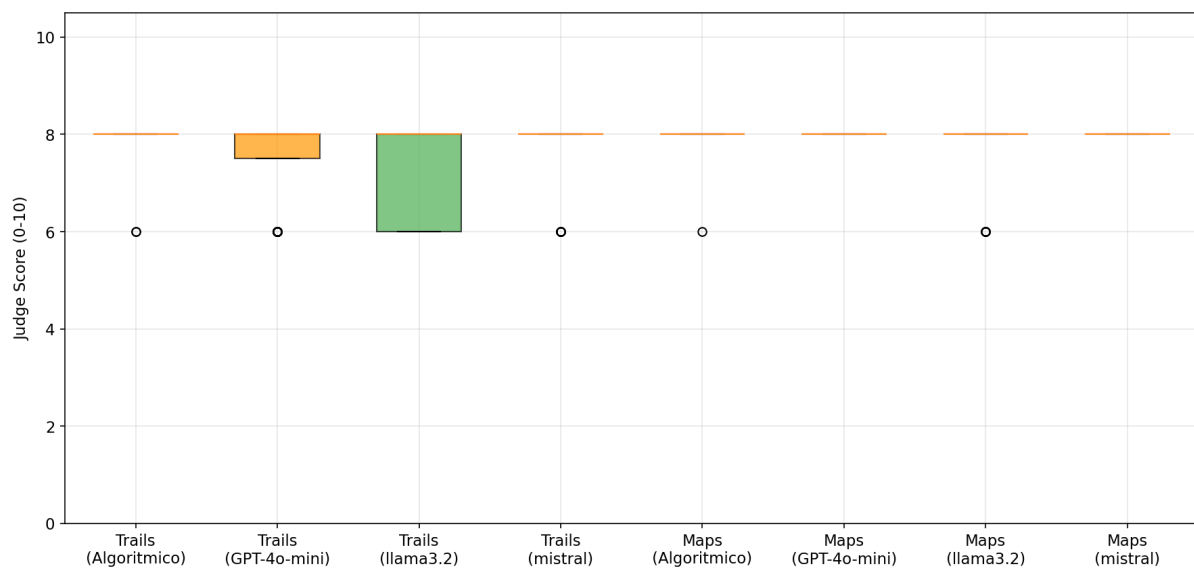


Figura 8. Box plot de judge scores por metodo.

4.5. Analisis de trade-offs calidad vs. eficiencia

La Tabla 13 presenta el analisis de trade-offs entre calidad y eficiencia.

Tabla 13. Trade-offs calidad vs. eficiencia.

Metodo	Coherencia	Tiempo medio (s)	Trade-off
Trails (Algoritmico)	0.694	0.1	Mejor calidad, mas rapido, determinista
Trails (GPT-4o-mini)	0.649	15.8	Alta calidad, requiere API externa y costo
Maps (Algoritmico)	0.581	0.1	Buena calidad, rapido, determinista
Maps (GPT-4o-mini)	0.516	17.9	Calidad moderada, costo de API
Trails (LLaMA 3.2)	0.500	4.6	Calidad variable, sin costo de API
Maps (LLaMA 3.2)	0.339	5.2	Baja calidad, tiempo moderado
Trails (Mistral)	0.347	3.0	Alta variabilidad, resultados inestables
Maps (Mistral)	0.223	3.7	Peor calidad general

El metodo Trails con scorer algoritmico ofrece el mejor equilibrio: mayor coherencia, menor tiempo de ejecucion y resultados deterministas. Los metodos con GPT-4o-mini son competitivos en calidad pero introducen dependencia de API externa y costos. Los modelos locales no logran competir con los metodos algoritmicos en coherencia medida por `avg_edge_coherence` sobre el corpus completo.

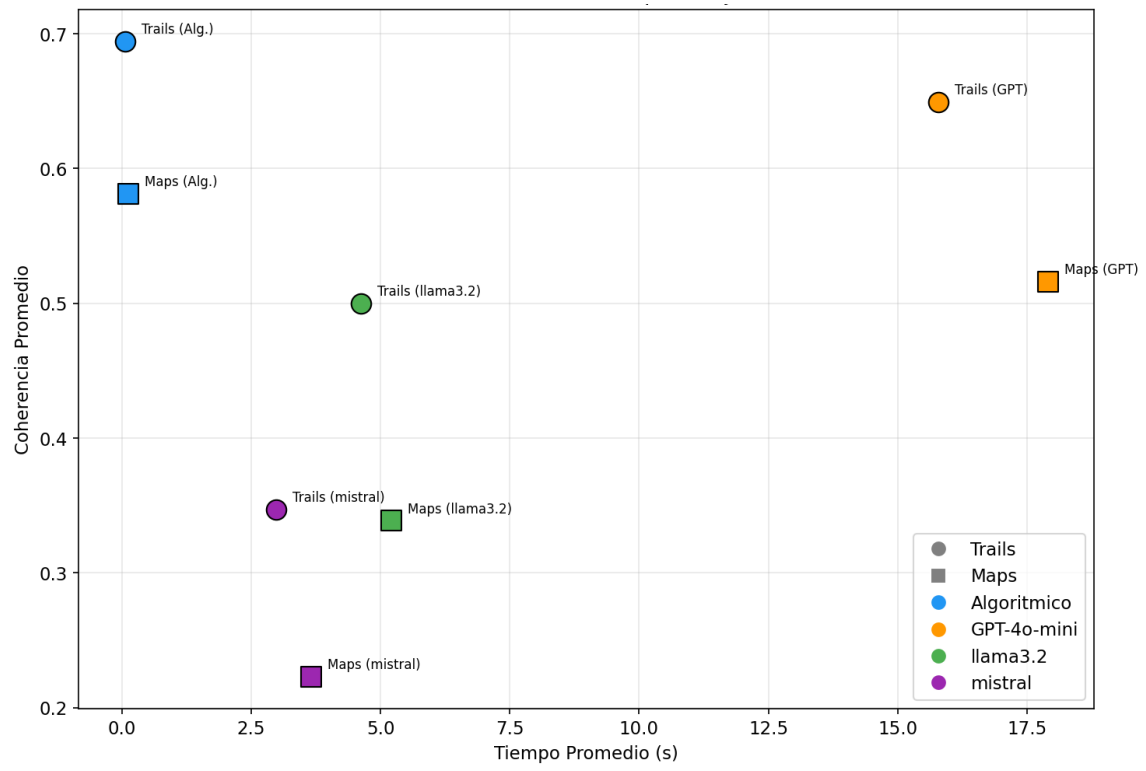


Figura 9. Diagrama de calidad vs. tiempo de ejecucion.

CAPITULO V: DISCUSIONES PRELIMINARES

El presente capitulo presenta las conclusiones derivadas del proyecto, el estado de cumplimiento de los objetivos y el trabajo restante.

5.1. Estado de cumplimiento de los objetivos

La Tabla 14 resume el estado de avance respecto a cada objetivo especifico.

Tabla 14. Estado de cumplimiento de los objetivos especificos.

Objetivo especifico	Estado	Evidencia
OE1. Identificar modelos locales viables	Cumplido	Se seleccionaron LLaMA 3.2 y Mistral 7B tras evaluar cuatro candidatos. Ambos ejecutables de forma estable en hardware limitado via Ollama.
OE2. Implementar pipeline experimental	Cumplido	Pipeline completo implementado: scoring (3 variantes), extraccion (2 algoritmos), evaluacion (LLM-as-a-judge), metricas y ejecucion batch con paralelismo.
OE3. Evaluar coherencia y eficiencia	Cumplido	384 experimentos ejecutados sobre corpus completo (931 articulos). Evaluacion estadistica completa con t-tests de Welch, correccion de Bonferroni y Cohen's d.
OE4. Analizar trade-offs	Cumplido	Analisis comparativo completado. Se identificaron los trade-offs entre calidad, estabilidad, velocidad y costo.

5.2. Hallazgos principales

Los resultados obtenidos permiten establecer las siguientes conclusiones:

Los metodos algoritmicos basados en embeddings (Sentence-Transformers) superan a los modelos de lenguaje locales en coherencia narrativa medida por avg_edge_coherence ($p < 0.001$, Cohen's $d > 2$). Este resultado se invierte respecto a los experimentos preliminares sobre el dataset curado de 15 documentos, lo que evidencia la importancia de validar sobre corpus completos y diversos.

Narrative Trails (MaxiMin) supera consistentemente a Narrative Maps (LP relaxation) en coherencia de aristas, independientemente del scorer utilizado. La ventaja de Trails es de +0.11 a +0.16 puntos en coherencia media, con significancia estadistica en la mayoria de comparaciones. Esto se explica porque el objetivo de maximizar la minima coherencia (Trails) produce caminos mas uniformemente coherentes que el objetivo de maximizar la coherencia total (Maps).

GPT-4o-mini es competitivo con el algoritmico en Narrative Trails (0.649 vs. 0.694, $p_{\text{bonferroni}} = 0.432$, sin diferencia significativa), lo que sugiere que el scorer propietario es una alternativa viable cuando se combina con el algoritmo de extraccion adecuado.

Los modelos locales (LLaMA 3.2 y Mistral 7B) presentan coherencia significativamente menor y alta variabilidad entre segmentos del corpus. LLaMA 3.2 supera a Mistral en todos los escenarios evaluados.

El protocolo LLM-as-a-judge con LLaMA 3.2 como juez muestra baja capacidad discriminativa: todos los metodos obtienen judge scores entre 7 y 8, sin reflejar las diferencias sustanciales en coherencia de aristas. Esto plantea interrogantes sobre la validez del juez para este tipo de evaluacion, o bien sugiere que la metrica avg_edge_coherence y el judge score capturan dimensiones diferentes de la calidad narrativa.

La coherencia de los metodos LLM se degrada significativamente al aumentar el tamano de la narrativa (n), mientras que los metodos algoritmicos mantienen estabilidad. Trails (LLaMA 3.2) pierde un 32% de coherencia al pasar de n=6 a n=18, en tanto que Trails (Algoritmico) solo pierde un 5%.

5.3. Dificultades encontradas

Restricciones de hardware: Los modelos Phi-3 y Gemma 2 fueron descartados debido a problemas de sobrecalentamiento del equipo durante ejecuciones prolongadas. La ejecucion final se realizo en Google Colab con GPU A100 para manejar el volumen de 384 experimentos.

Divergencia entre metricas: La discrepancia entre avg_edge_coherence y judge scores dificulta la interpretacion univoca de los resultados. La coherencia de aristas favorece a los metodos algoritmicos, mientras que el juez LLM no discrimina significativamente entre metodos.

Variabilidad de los LLMs locales: Los scorers basados en LLM producen matrices de coherencia mas dispersas que el scorer de embeddings, lo que resulta en alta variabilidad entre segmentos del corpus y entre tamanos de narrativa.

5.4. Trabajo restante

Las actividades pendientes para completar el proyecto son:

1. Redaccion de conclusiones finales (actividad 15): Profundizacion del analisis de la divergencia entre metricas y discusion de las implicaciones para el campo.
2. Redaccion del informe final (actividad 16): Integracion de todos los resultados y redaccion del documento final. Fecha limite: 16 de junio de 2026.

Referencias Bibliograficas

- Brown, T. B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., et al. (2020). Language models are few-shot learners. *Advances in Neural Information Processing Systems*, 33, 1877-1901.
- Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. *Proceedings of NAACL-HLT 2019*, 4171-4186.
- German, F., Keith, B., & North, C. (2025). Narrative trails: A method for coherent storyline extraction via maximum capacity path optimization. *Text2Story 2025 Workshop, CEUR-WS Vol-3964*.
- Jiang, A. Q., Sablayrolles, A., Mensch, A., Bamford, C., Chaplot, D. S., Casas, D. D. L., et al. (2023). Mistral 7B. *arXiv preprint arXiv:2310.06825*.
- Keith, B. (2025). LLM-as-a-judge approaches as proxies for mathematical coherence in narrative extraction. *Electronics*, 14(13), 2735.
- Keith, B. F., & Mitra, T. (2021). Narrative maps: An algorithmic approach to represent and extract information narratives. *Proceedings of the ACM on Human-Computer Interaction*, 4(CSCW3).
- Keith, B. F., Mitra, T., & North, C. (2023). A survey on event-based news narrative extraction. *ACM Computing Surveys*, 55(14s).
- Li, M., Ma, T., Yu, M., Wu, L., Gao, T., Ji, H., & McKeown, K. (2021). Timeline summarization based on event graph compression via time-aware optimal transport. *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing (EMNLP 2021)*, 6443-6456.
- Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence embeddings using Siamese BERT-networks. *Proceedings of EMNLP-IJCNLP 2019*, 3982-3992.
- Shahaf, D., & Guestrin, C. (2010). Connecting the dots between news articles. *Proceedings of the 16th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD '10)*.
- Touvron, H., Lavril, T., Izacard, G., Martinet, X., Lachaux, M.-A., Lacroix, T., et al. (2023). LLaMA: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2017). Attention is all you need. *Advances in Neural Information Processing Systems*, 30, 5998-6008.
- Zheng, L., Chiang, W.-L., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., Lin, Z., Li, Z., Li, D., Xing, E. P., Zhang, H., Gonzalez, J. E., & Stoica, I. (2023). Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. *Advances in Neural Information Processing Systems*, 36.

Anexo A: Estructura del Proyecto y Formato de Resultados

A.1. Estructura del repositorio

```
narrative-extraction/
|-- pipeline.py          # Pipeline principal (CLI)
|-- data/
| |-- convert.py        # Conversor FINAL_DATA.json -> corpus.jsonl
| |-- sample.jsonl      # Dataset experimental (15 documentos)
| |-- corpus.jsonl      # Dataset completo (931 articulos)
|-- baselines/
| |-- result.py         # NarrativeResult dataclass
| |-- maps.py           # Narrative Maps (LP relaxation)
| |-- trails.py         # Narrative Trails (MaxiMin)
|-- scoring/
| |-- algorithmic.py    # Scorer embeddings (Sentence-Transformers)
| |-- precompute.py     # Cache de matrices de coherencia
|-- llm/
| |-- scorer.py         # Scorer LLM local (Ollama)
| |-- openai_scorer.py  # Scorer propietario (OpenAI)
|-- evaluation/
| |-- judge.py          # LLM-as-a-Judge
|-- metrics/
| |-- recorder.py       # Wall time, memoria, coherencia
|-- experiments/
| |-- run_batch.py      # Ejecutor batch de experimentos
|-- analysis/
| |-- statistics.py     # Tests estadísticos
| |-- plots.py          # Generación de gráficos
| |-- plots/            # Gráficos generados (PNG)
|-- results/            # Resultados de experimentos (JSON)
```

A.2. Formato de resultados

Adicionalmente al repositorio Python descrito en A.1, se implementó un notebook de Google Colab orientado a aprovechar la GPU NVIDIA A100 disponible en el entorno de Colab, dado que la ejecución masiva de 384 experimentos excedía la capacidad del hardware de desarrollo local (Apple M4 Pro sin GPU dedicada). El notebook actúa como orquestador del repositorio: se encarga de configurar el entorno de ejecución, clonar el código desde GitHub, instalar dependencias y lanzar el batch experimental con paralelismo de 8 workers.

La estructura del notebook se compone de las siguientes secciones principales:

1. Verificación del entorno: confirmación de disponibilidad de GPU mediante el comando `nvidia-smi` y validación de la versión de Python y librerías preinstaladas en Colab.
2. Configuración de variables y secretos: definición de variables de entorno, incluyendo la API key de OpenAI requerida para el scorer propietario, almacenada como secret de Colab por seguridad.
3. Clonado del repositorio: descarga del código fuente desde GitHub mediante `git clone`, fijando la rama o commit a evaluar para garantizar la reproducibilidad de los resultados.

4. Instalacion de dependencias: instalacion de los paquetes listados en requirements.txt, incluyendo sentence-transformers, PuLP, ollama, openai, scipy, pandas, networkx y matplotlib.
5. Instalacion y arranque de Ollama: descarga e instalacion del binario de Ollama en el entorno Colab, levantamiento del servicio en segundo plano y descarga (ollama pull) de los modelos LLaMA 3.2 y Mistral 7B.
6. Ejecucion del batch experimental: invocacion de experiments/run_batch.py con los parametros del diseno experimental (tamanos de narrativa n=6, 12, 18; puntos de inicio en indices 0, 50, 100 y 150; cinco repeticiones por configuracion para los metodos basados en LLM; 8 workers en paralelo).
7. Persistencia de resultados: sincronizacion de los archivos JSON generados con Google Drive para evitar la perdida de resultados en caso de desconexion de la sesion de Colab.
8. Analisis exploratorio: ejecucion de los modulos de analisis (analysis/statistics.py, analysis/plots.py) para generar las tablas estadisticas y los graficos incluidos en el Capitulo IV.

Esta arquitectura desacoplada (repositorio modular + notebook orquestador) permite mantener el codigo del pipeline reproducible y portable, mientras que el notebook concentra los pasos especificos del entorno Colab y de la configuracion experimental.

A.3. Formato de resultados

Cada experimento genera un archivo JSON con la siguiente estructura:

```
{
  "method": "maps_llm",
  "model": "mistral",
  "n": 6,
  "window_days": 30,
  "narrative": [
    {"id": "doc_01", "date": "2024-01-15", "title": "..."}
  ],
  "metrics": {
    "wall_time_seconds": 131.8,
    "peak_memory_mb": 2.0,
    "avg_edge_coherence": 0.88
  },
  "judge": {
    "coherence_score": 9,
    "justification": "...",
    "evaluation_type": "pointwise"
  },
  "graph": {
    "nodes": [...],
    "edges": [{"source": "doc_01", "target": "doc_05", "coherence_score": 0.85}]
  }
}
```